

Rybina search (rybina)

Op de Schueberfouer, de grootste kermis in Luxemburg, is er een populair getallen-raadspel. De showman, Rybina, verbergt een geheim wachtwoord van de vorm AB , waarbij A en B strings met getallen zijn van N cijfers. Hij laat je het wachtwoord raden, en na elke gok vertelt hij of je gok te hoog of te laag is.

Na elke gok kan Rybina echter de twee helften van het wachtwoord omwisselen. Dan wordt bijvoorbeeld AB veranderd in BA (of BA wordt AB). Hij vertelt je ook of hij dit gedaan heeft. Je volgende gok wordt dan vergeleken met het omgewisselde wachtwoord.

Jouw taak is om het geheime wachtwoord te achterhalen met maximaal 64 gokken.

Implementatie

Je moet een enkel `.cpp` bronbestand inleveren.



Bij de bijlagen van deze opgave vind je een template `rybina.cpp` met een voorbeeldimplementatie.

Je moet de volgende functie implementeren:

C++

```
void rybina_search(int N)
```

- De integer N is het aantal cijfers in elke helft van het wachtwoord.

Je programma kan de volgende functie aanroepen, die al gedefinieerd is in de grader:

C++

```
GuessResult guess(long long X)
```

- De integer X is het wachtwoord dat je als gok hebt ingevoerd. Het moet niet-negatief zijn en mag maximaal $2N$ cijfers bevatten wanneer het decimaal uitgeschreven wordt (dus $0 \leq X < 10^{2N}$).
- Als je gok correct is, wordt je programma beëindigd en ontvang je het **accepted** resultaat voor het huidige testgeval.
- Anders geeft de functie een `GuessResult` terug, een struct met integer veld `comparison` en een boolean veld `swapped`.
- De waarde van `comparison` is 1 als je gok te laag was en -1 als je gok te hoog was.
- De waarde van `swapped` geeft aan of het wachtwoord is omgewisseld nadat je gok gemaakt werd.

Merk op dat de grader **adaptief** is en het juiste antwoord of wanneer er wordt gewisseld voor een testgeval dus niet vooraf vastliggen.

Voorbeeld Grader

Bij de bijlagen van deze opgave vind je een vereenvoudigde versie van de grader die tijdens de beoordeling wordt gebruikt, die je kunt gebruiken om je oplossingen lokaal te testen. De voorbeeldgrader leest data uit `stdin` en roept de functie `rybina_search` aan. Hierbij wordt het volgende invoerformaat gebruikt.

De invoer bestaat uit twee regels:

- Regel 1: de integer N .
- Regel 2: twee integers S en T . De eerste, S , is de seed die gebruikt wordt om de willekeurige wisselingen te bepalen; de tweede, T , is het correcte antwoord: het initiële wachtwoord AB geschreven als een enkele integer met $0 \leq T < 10^{2N}$.

Na elke gok wisselt de voorbeeldgrader de twee helften van het wachtwoord willekeurig om, met behulp van de seed S . (De grader die tijdens de officiële beoordeling wordt gebruikt, is in plaats daarvan adaptief.)

De voorbeeldgrader geeft de uitkomst via `stderr`: wanneer je oplossing het wachtwoord raadt, print de grader het aantal gebruikte gokken, en anders een korte melding met de reden (een ongeldige gok of te veel gokken).

Randvoorwaarden

- $1 \leq N \leq 9$.

Je kunt de functie `guess` maximaal 64 keer aanroepen.

Scoring

Je programma wordt getest op meerdere testgevallen gegroepeerd in subtaken. Om de score van een subtaak te behalen, moet je programma alle testgevallen correct oplossen.

- **Subtask 0 [0 punten]**: Voorbeeldtestgevallen.
- **Subtask 1 [5 punten]**: $N = 1$ en $B = 0$, d.w.z. de tweede helft van het initiële wachtwoord is 0.
- **Subtask 2 [5 punten]**: $N = 1$.
- **Subtask 3 [10 punten]**: $N \leq 4$ en Rybina zal het wachtwoord nooit omwisselen.
- **Subtask 4 [20 punten]**: Rybina zal het wachtwoord nooit omwisselen.
- **Subtask 5 [15 punten]**: Rybina zal het wachtwoord altijd omwisselen.
- **Subtask 6 [20 punten]**: $N \leq 4$
- **Subtask 7 [25 punten]**: Geen aanvullende beperkingen.

Voorbeelden

Grader	Oplossing
<code>rybina_search(2)</code>	
	<code>guess(1000)</code>
<code>return [1, 1]</code>	
	<code>guess(1000)</code>
<code>return [-1, 0]</code>	
	<code>guess(412)</code>
<code>return [0, 0]</code>	

Uitleg

In het eerste voorbeeld is $N = 2$ en zijn de twee delen van het geheime wachtwoord 12 en 04. Er wordt een gok van 1000 gedaan; op dit moment is het wachtwoord 1204, dus de gok is te laag en na de gok meldt Rybina dat het wachtwoord is omgewisseld. Het wachtwoord is nu 412 en dezelfde gok van 1000 wordt gedaan. Dit keer is de gok te hoog en Rybina meldt ook dat het wachtwoord daarna niet is omgewisseld. Ten slotte wordt het wachtwoord 412 geraden, wat correct blijkt en het programma is klaar.